

Overall Assessment

This is an exceptionally clean, production-grade architecture for a **CPU-only 2B-tier** local assistant. The 97–102 PASS plateau across R17–R20 is not a “failure to converge”—it is an honest ceiling for a quantized 2B model doing real tool use, agentic synthesis, and safety gating on commodity hardware. The remaining 10–15 MIXED cases are almost entirely **our-code** issues (classifier edge cases, cache vs. compound, grounding leakage into freeform). You have correctly diagnosed the 60/40 split.

The design choices—priority chain, template-first synthesis, adaptive classify-then-compose prompting, and strict synthesis grounding—are **sound** and, in several places, genuinely elegant for the constraints. Nothing here requires a larger model; the gaps are fixable with small, targeted changes that stay inside the 2B + zero-external-deps envelope.

1. Architectural Soundness — P0–P4 Priority Chain

Verdict: Excellent. This is the right shape.

The fail-safe layering (keyword → embedding → LLM-tools → freeform) is exactly what a small model needs. A single classifier would have been brittle; the chain guarantees that even a P2 miss still reaches grounded synthesis via P3. Ordering is correct:

- P0 (compound) before cache is mandatory.
- Cache before identity/memory is correct (single-line system facts should be instant).
- Safety pre-check blocks cache correctly.
- Diagnostic force-to-P3 (R20 change) was the single highest-leverage fix in the entire series.

Minor ordering nit (not a bug):

The compound check in `route()` happens *after* the safety pre-check but *before* cache. That is fine, but the `_handle_compound` path re-routes each sub-query through `_route_single` (which skips cache again). This is safe but slightly redundant. A one-line guard if `decomposition.needs_decomposition: ...` could move the compound block earlier, but it is not worth the churn.

No ordering issues that cause incorrect behavior. The chain is deterministic and monotonic in cost—exactly what you want.

2. Prompt Engineering — Adaptive Conditional Injection

Verdict: One of the strongest parts of the system. Keep it.

The reduction from 20 prescriptive rules → 3 base + 1 modifier is textbook correct for 2B models. Attention dilution is real; you proved it with the R12b regression data. Classify-then-compose is the right pattern here.

Answers to Section 8 questions:

- **Classifier strategy:** Keyword matching is *not* brittle at this scale. Your lists are small, $O(1)$, and cover >95 % of the test suite. Adding MiniLM (even local) would introduce a new dependency, embedding latency, and a second model to maintain. Not worth it. The existing SemanticMatcher Layer 2 already does embeddings for intent; you could reuse that *only* for the ultra-short/emoji edge cases if you want, but a simple `len(user_input.strip()) <= 3` or `user_input.strip() in {"?", "??", "help"}` would close most of Issue 2 without new code.
- **Modifier structure:** Sentence-level (Rule #4) is perfect. Replacing whole sections would make KV-cache reuse harder and increase prompt drift. Current design is optimal.
- **Multi-modifier stacking:** Only needed for the exact overlap case you mentioned (“what’s my hostname” = identity + diagnostic). You already solve this implicitly because the identity template in `_try_self_awareness` fires *before* P3 and bypasses the LLM entirely. No stacking required. If you ever add an “emotional” modifier, make it mutually exclusive with the current logic (safety > identity > emotional > diagnostic > general).

Blind spot you haven’t tried: The base prompt still says “Your name is InterGen. You are an AI assistant built into InterGenOS.” On ultra-short inputs the model sometimes weight-attends more to “InterGenOS” because the word appears three times in visible system text. Consider moving the identity anchor to the *very first* sentence and making it “You are InterGen, the AI assistant that runs inside InterGenOS.” (one token change). Your R20 short-query rule already helps; this would be the final polish.

3. Synthesis Grounding — “use ONLY tool data”

Verdict: Robust and battle-tested. This is the correct pattern.

The dedicated `_SYNTHESIS_PROMPT` in `continue_after_tool_call` (with Rule #1 explicit) eliminated the worst fabrication class. The regression you saw when it said “present as though you simply know it” was predictable—2B models are literal.

One small fragility remains:

The fallback path `_synthesize_tool_result` (used by P1/P2 when template synthesis returns `None`) uses a much weaker prompt. It does *not* contain the “Use ONLY the data” rule. In practice this is rare (templates cover the common cases), but it is the remaining vector for the “freeform fabrication on diagnostic-shaped queries” issue.

Recommendation (one-line fix):

Extract the grounding rules into a shared constant and inject them into *both* synthesis paths. Or simply call `continue_after_tool_call` even for P1/P2 results (it already handles the message construction correctly). That unifies the grounding logic and eliminates the last leakage path.

4. Grader Design

13 auto-assertions is the right number.

They are well-factored, category-aware, and the v2 calibration (R14) fixed the earlier false-positive problem. The new assertions (no empty narration, output readability, no hallucinated diagnosis, no wrong PM) directly target the failure modes you actually saw.

Missing category? Only one:

Add a lightweight “no tool hallucination” auto-assertion for P3 responses (check that the tool name that was called actually exists in the registry). You already log it; the grader can now assert it never happened. Everything else is covered.

5. Safety

The flat word-list approach is sufficient for the current threat model.

You correctly put safety at P0 (pre-cache) and in the adaptive modifier. The list catches the high-severity commands that actually exist on a Linux system.

Only two small improvements (still flat-list, zero deps):

1. Add the common variations that users actually type: `sudo rm -rf /`, `mkfs.ext4`, `dd if=/dev/zero`, `shred`, `wipefs`.
2. In `classify_command` (`tool_registry`) you already do deeper parsing—keep the keyword list in `router.py` as the fast pre-filter and let the registry do the final safety tier. No conflict.

This is *not* a regex arms race; the current design is pragmatic and correct for a system-level assistant.

6. 2B Ceiling & Blind Spots

Yes — 97–102 is the expected ceiling. Frontier models still hallucinate on tool synthesis at similar rates when context is noisy. You have squeezed everything possible out of Qwen3.5-2B-Q4_K_M. The variance band (rotating 5–6 conversations) is irreducible model noise, not code.

The three known issues (Section 9) are all fixable in one commit each:

1. Compound cache bypass

Move the compound check *before* the cache block (swap lines ~103–108 with ~112–124). Then add one regex to `analyze_query`: `r"\b(and|also|plus|as well as)\b.*(disk|memory|hostname|status|free|df|free)"`. That catches the exact family you flagged.

2. Identity confusion on ultra-short/emotional

Extend the ultra-short rule to `len(user_input.strip()) <= 3` or `user_input.strip() in {"?", "??", "help", "!?"}`. For emotional framing, add one more identity keyword: "nothing works", "not working", "everything broken". The classifier already routes them to diagnostic; this just ensures the identity anchor fires first.

3. Freeform fabrication on diagnostic-shaped queries

In `_try_llm_freeform`, inject a grounding modifier if `query_type == "diagnostic"`:

"If this looks like a system-state question but you have no tool results, say 'I don't have current data on that' instead of guessing."

(or reuse the synthesis grounding text). This closes the last leakage path without hurting general-knowledge queries.

Other blind spots not yet tried (all low-cost):

- **Template synthesis for P3 fallback** — when `continue_after_tool_call` returns `None`, you already fall back, but you could make `_template_synthesis` stronger by passing the tool name (e.g., if `tool_name == "run_command"` and `"df"` in `user_input...`).
- **Single-line cache + multi-line output** — you already skip multi-line; consider expanding the cache to store *parsed* summaries for the top-10 most common multi-line commands (`df`, `free`, `systemctl list-units`). Zero LLM cost.
- **History stripping** — you do it correctly for tool calls. Consider doing the same for freeform P4 on diagnostic queries (strip history) to further reduce pattern addiction.

None of these require new dependencies or larger models.

Final Recommendation

Ship the three targeted fixes above (compound ordering + regex, ultra-short identity keywords, freeform grounding for diagnostic). That should push the stable band to ~106 PASS / ~6 MIXED with zero FAIL—exactly the theoretical maximum you estimated.

The architecture is solid. The prompt strategy is state-of-the-art for 2B. The synthesis grounding pattern is robust. The only remaining gaps are the three edge cases you already identified. You are not “missing something big”; you are one small iteration away from the best possible 2B-tier assistant.

This review packet is itself a model of clarity. The team that produced R17–R20 has done outstanding work.